Dynamic Branch Prediction

Sparsh Mittal

Courtesy for some slides: Dr. Onur Mutlu

Some Direction Prediction Techniques

- Static Predictors (Compile time)
  - Always not taken
  - Always taken
  - BTFN (Backward taken, forward not taken)

- Dynamic Predictors (Run time)
  - One-level Predictor
    - One-bit Predictor (also called last time predictor)
    - Two-bit Predictor
  - Two-level predictor
    - Global History Predictor
    - Local History Predictor

Dynamic Branch Prediction

- Idea: Predict branches based on dynamic information (collected at run-time)

- Advantages
  + Prediction based on history of the execution of branches
    + It can adapt to dynamic changes in branch behavior

- Disadvantages
  -- More complex than static predictors (requires additional hardware)

Our first example of dynamic branch predictor

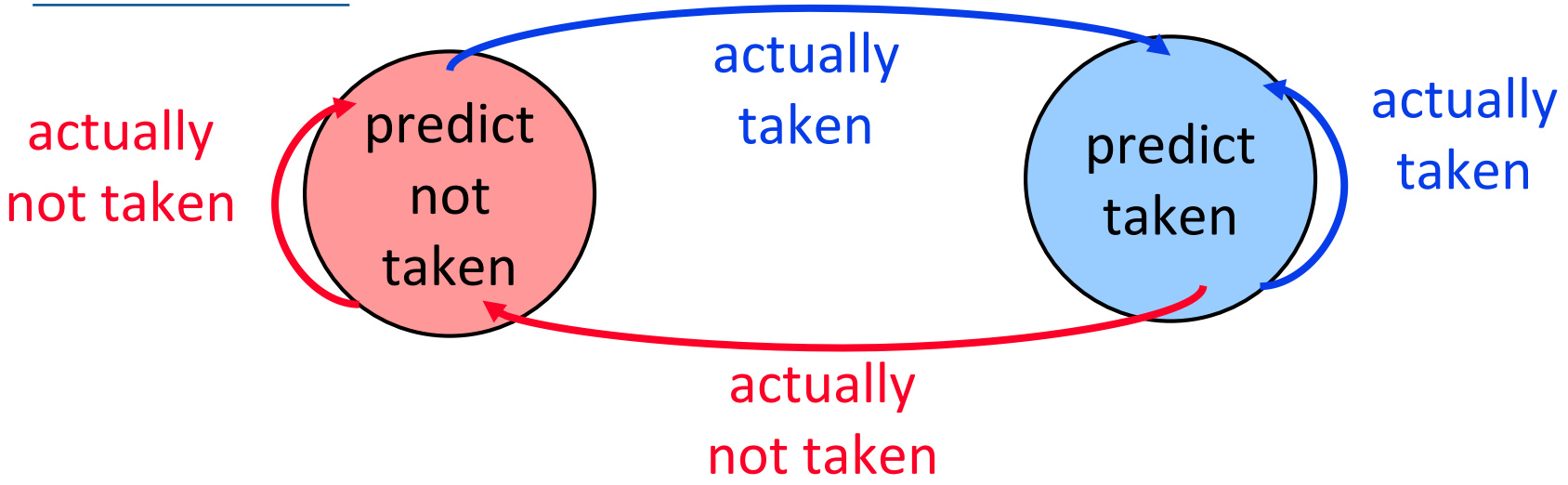
Last-time Predictor (also called one-bit predictor)

4

Last Time Predictor

- It uses one bit per branch
- This bit records which direction branch went last time it executed
- The same direction is output as the prediction next time

State machine:



Solved Example 1

- (a) Consider this sequence: T, NT, NT, NT, T
- What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
- Always-Taken Predictor:
- Actual T NT NT NT T
- Predict T T T T T
- Accuracy = 2/5 = 0.4

- Always-not-taken Predictor:
- Actual T NT NT NT T
- Predict NT NT NT NT NT
- Accuracy = 3/5 = 0.6

Solved Example 1 (continued)

What is the accuracy of a last-time predictor for the first 5 branches in this sequence? Assume this predictor starts in the "Predict not taken" state.

Actual	T	NT	NT	NT	T
Predict	NT	T	NT	NT	NT

Accuracy = 2/5 = 0.4

Example 2

- For(int i=0; i<N; i++) {
- if(i>N/2) doSomething;
- }
- This code is executed once. The last-time predictor for this "if" branch is initialized to "not-taken". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	NT	NT	NT
1	NT	NT	NT
2	NT	NT	NT
3	NT	NT	NT
4	NT	T	T
5	T	T	T

Accuracy = 5/6

Example 3

- For(int i=0; i<N; i++) {
- if(i%2==0) doSomething;
- }
- This code is executed once. The last-time predictor for this "if" branch is initialized to "not-taken". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	NT	T	T
1	T	NT	NT
2	NT	T	T
3	T	NT	NT
4	NT	T	T
5	T	NT	NT

Accuracy = 0/6

For loop involves conditional branch

- Consider the below for loop-code:

```
For (int j=0; j<N; j++) {
doSomething;
}
ExecuteInstructionAfterForLoop;
```

- Above for loop can be written as:

```
        j=0;
Loop: if(j>=N) goto Exit   ←——— Conditional branch
        doSomething;
        j++
        goto Loop
Exit:  ExecuteInstructionAfterForLoop;
```

Example 4: Nested *for* loop

- For (int i=0; i<M; i++) {
- For (int j=0; j<N; j++) {
- doSomething;
- }
- }
- In second for loop, we check j<N condition, which is a branch instruction.
- We use last-time predictor for this branch, which is initialized to T.
- Assume: Going inside loop= taken. Otherwise =not-taken.
- N=6
- Compute accuracy of this branch predictor in steady-state.

Example 4 (continued)

Initially, i=0; we enter into second for loop for the first time.

This "T" comes from fact that the predictor is initialized to T.

j	Prediction	Outcome	NewState
0	T	T	T
1	T	T	T
2	T	T	T
3	T	T	T
4	T	T	T
5	T	T	T
6	T	NT	NT

Saved for next time

Accuracy = 6/7

Example 4 (continued)

Now, i=1; we enter into second for loop for the second time.

This "NT" comes from NewState value stored from first time.

j	Prediction	Outcome	NewState
0	NT	T	T
1	T	T	T
2	T	T	T
3	T	T	T
4	T	T	T
5	T	T	T
6	T	NT	NT

Accuracy = 5/7. Thus, steady-state accuracy is 5/7.

Summary

- Always mispredicts the last iteration and the first iteration of a loop branch (N= number of iterations)
  - Accuracy for a loop with N iterations = (N-1)/(N+1)

+ Good for loop branches for loops with large N

-- Poor for loop branches for loops will small N

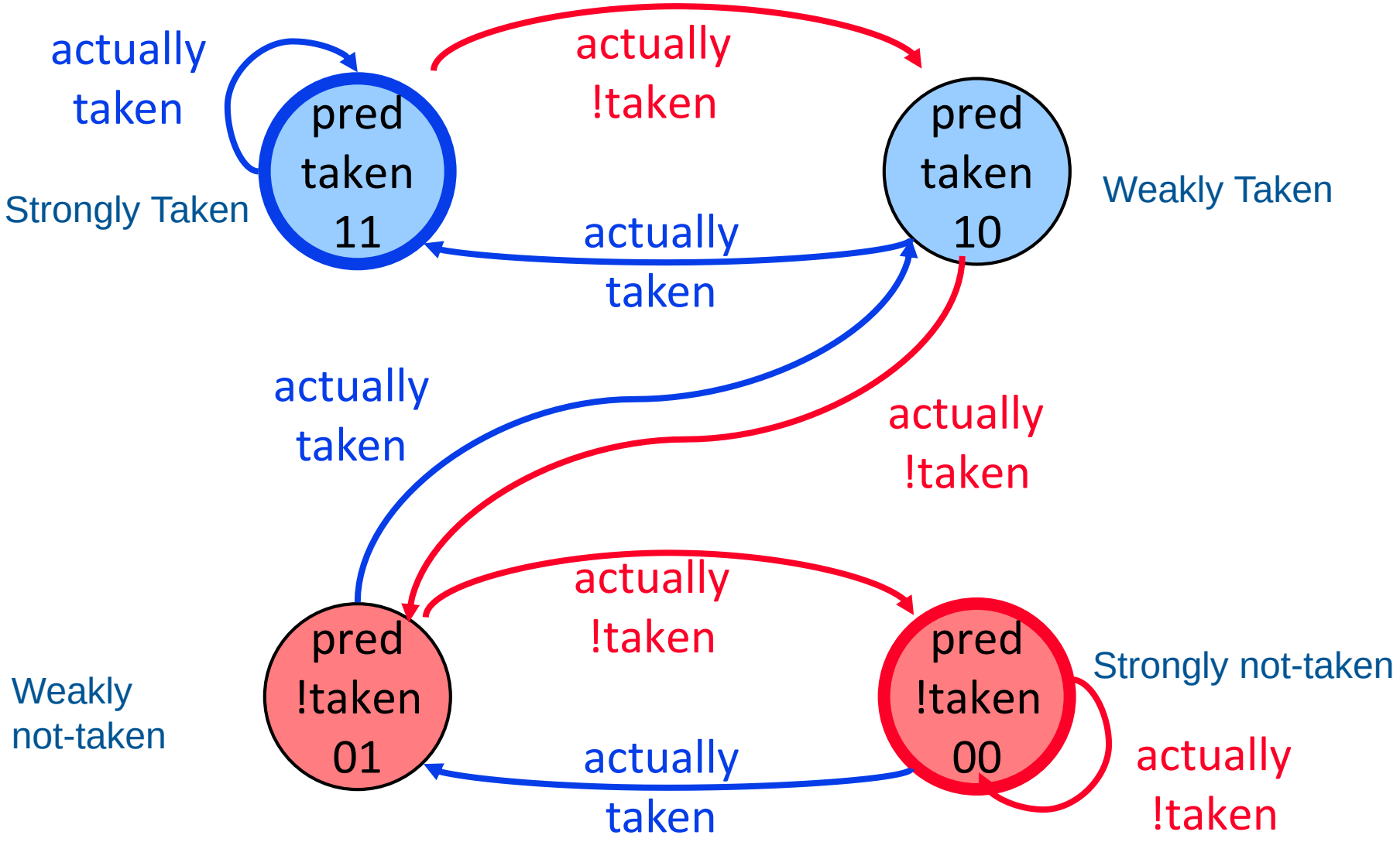
- Our second example of dynamic branch predictor

Two-bit predictor

Improving the Last Time Predictor

- Problem: A last-time predictor changes its prediction from T→NT or NT→T too quickly
  - even though the branch may be mostly taken or mostly not taken
- Solution: Add hysteresis to the predictor so that prediction does not change on a single different outcome
  - Use two bits to track the history of predictions for a branch instead of a single bit
  - Can have 2 states for T or NT instead of 1 state for each
- One more bit provides hysteresis
- A strong prediction does not change with one single different outcome

State Machine for 2-bit Saturating Counter



Example 1a: initialized with Weak NT

- For(int i=0; i<N; i++) {

  if(i%2==0) doSomething;

  }
- This code is executed once. The two-bit predictor for this "if" branch is initialized to "weak NT". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	NT	T	Weak T
1	T	NT	Weak NT
2	NT	T	Weak T
3	T	NT	Weak NT
4	NT	T	Weak T
5	T	NT	Weak NT

Accuracy = 0/6

Example 1b: initialized with Weak T

- For(int i=0; i<N; i++) {

  if(i%2==0) doSomething;

  }
- This code is executed once. The two-bit predictor for this "if" branch is initialized to "weak T". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	T	T	Strong T
1	T	NT	Weak T
2	T	T	Strong T
3	T	NT	Weak T
4	T	T	Strong T
5	T	NT	Weak T

Accuracy = 3/6

Example 1c: initialized with Strong T

- For(int i=0; i<N; i++) {
- if(i%2==0) doSomething;
- }
- This code is executed once. The two-bit predictor for this "if" branch is initialized to "Strong T". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	T	T	Strong T
1	T	NT	Weak T
2	T	T	Strong T
3	T	NT	Weak T
4	T	T	Strong T
5	T	NT	Weak T

Accuracy = 3/6

Example 1d: initialized with Strong NT

- For(int i=0; i<N; i++) {
- if(i%2==0) doSomething;
- }
- This code is executed once. The two-bit predictor for this "if" branch is initialized to "strong NT". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	NT	T	Weak NT
1	NT	NT	Strong NT
2	NT	T	Weak NT
3	NT	NT	Strong NT
4	NT	T	Weak NT
5	NT	NT	Strong NT

Accuracy = 3/6

Example 2

- For(int i=0; i<N; i++) {
- if(i>N/2) doSomething;
- }
- This code is executed once. The two-bit predictor for this "if" branch is initialized to "strongly not-taken". Find accuracy. N=6

i	Prediction	Outcome	NewState
0	NT	NT	Strong NT (sat)
1	NT	NT	Strong NT (sat)
2	NT	NT	Strong NT (sat)
3	NT	NT	Strong NT (sat)
4	NT	T	Weak NT
5	NT	T	Weak T

Accuracy = 4/6

Two-Bit Counter (2BC) Based Prediction

- Accuracy for a loop with N iterations = (N)/(N+1)
- By contrast, 1-bit counter had accuracy of (N-1)/(N+1)

+ Better prediction accuracy
-- More hardware cost

Example : Nested *for* loop

- For (int i=0; i<M; i++) {
- For (int j=0; j<N; j++) {
- doSomething;
- }
- }
- In second for loop, we check j<N condition, which is a branch instruction.
- We use 2-bit predictor for this branch, which is initialized to sT.
- Assume: Going inside loop= taken. Otherwise =not-taken.
- N=6
- Compute accuracy of this branch predictor in steady-state.

Example (continued)

Initially, i=0; we enter into second for loop for the first time.

This "T" comes from fact that the predictor is initialized to sT.

j	State	Prediction	Outcome	NewState
0	sT	T	T	sT
1	sT	T	T	sT
2	sT	T	T	sT
3	sT	T	T	sT
4	sT	T	T	sT
5	sT	T	T	sT
6	sT	T	NT	wT

Saved for next time

Accuracy = 6/7

Example (continued)

Now, i=1; we enter into second for loop for the second time.

This "T" comes from NewState value of wT stored from first time.

j	State	Prediction	Outcome	NewState
0	wT	T	T	sT
1	sT	T	T	sT
2	sT	T	T	sT
3	sT	T	T	sT
4	sT	T	T	sT
5	sT	T	T	sT
6	sT	T	NT	wT

Accuracy = 6/7. Thus, steady-state accuracy is 6/7.

Question

- Consider this sequence: T, NT, NT, NT, T
- What is the accuracy of a 2-bit predictor for the first 5 branches in this sequence, assume this predictor starts in the "Strongly not taken" state.
-
- Actual T NT NT NT T
- PredictorState 0 1 0 0 0
- Predict NT NT NT NT NT
- Accuracy = 3/5 = 0.6
- (Here predictor state 0/1/2/3 refer to stronglyNT / weaklyNT/ weaklyT/ stronglyT, respectively.)
-

Question (Continued)

- What is the accuracy of a 2-bit predictor if this pattern is repeated forever? Assume this predictor starts in the "Strongly not taken" state.
-
- Actual T NT NT NT T | T NT NT NT T | T NT NT NT T |
- Predictor 0 1 0 0 0 | 1 2 1 0 0 | 1 2 1 0 0 |
- Predict NT NT NT NT NT | NT T NT NT NT | NT T NT NT NT |
- Steady-state Accuracy = 2/5 = 0.4
-

Question

- What is the accuracy of a 2-bit predictor for the first 5 branches in this sequence? Assume this predictor starts in the "Strongly not taken" state.
-
- Actual T NT T NT T
- Predictor 0 1 0 1 0
- Predict NT NT NT NT NT
- Accuracy = 2/5 = 0.4

- What is the accuracy of a 2-bit predictor if this pattern is repeated forever? Assume this predictor starts in the "Strongly not taken" state.
-
- (Left for students to complete)